Федеральное государственное автономное образовательное учреждение высшего образования
Национальный исследовательский университет ИТМО

Факультет программной инженерии и компьютерной техники
Направление подготовки 09.03.04 Программная инженерия
Дисциплина «*Тестирование программного обеспечения*»

Отчёт

По лабораторной работе №1

Вариант: 408256

Студенты:

Беля А. И.

Хачатрян Г. А.

группа *P3315*

Преподаватель:

Демурчан В. В.

г. Санкт-Петербург, 2026 г.

Описание задания

1. Для указанной функции провести модульное тестирование разложения функции в степенной ряд. Выбрать достаточное тестовое покрытие.
2. Провести модульное тестирование указанного алгоритма. Для этого выбрать характерные точки внутри алгоритма, и для предложенных самостоятельно наборов исходных данных записать последовательность попадания в характерные точки. Сравнить последовательность попадания с эталонной.
3. Сформировать доменную модель для заданного текста. Разработать тестовое покрытие для данной доменной модели

Вариант **408256**

1. Функция **tg(x)**
2. Программный модуль для работы с хеш-таблицей с закрытой адресацией (Hash Integer, http://www.cs.usfca.edu/~galles/visualization/OpenHash.html)
3. Описание предметной области:

Например, в тот самый момент, когда Артур произнес "А у меня, кажется, большие проблемы с образом жизни", в ткани пространства-времени открылась случайная дыра и перенесла его слова далеко-далеко во времени через почти бескрайние просторы космоса в далекую галактику, где странные воинственные существа балансировали на грани ужасной межзвездной войны.

Выполнение

Исходный код

https://github.com/aeshabb/tpo_lab1

Задание 1: tg*(x)*

Функция tg(x) раскладывается в ряд Тейлора:

$\tan(x) = x + x^3/3 + 2x^5/15 + 17x^7/315 + 62x^9/2835 + 1382x^{11}/155925 + ...$

1. **Выберем “интересные” для тестирования точки:**

0.0 — должно определяться как 0 ровно, а не как какое-то приближение (как в эталонной реализации Math.tan);
π/6 ≈ 0.5236 — стандартное табличное значение, tg(π/6) = 1/√3 ≈ 0.5774, точка далеко от угловых случаев;

π/4 ≈ 0.7854 — стандартное табличное значение, tg(π/4) = 1 ровно;

π/3 ≈ 1.0472 — стандартное табличное значение, tg(π/3) = √3 ≈ 1.7321, близко к границе области сходимости;

−π/6, −π/4, −π/3 — симметричные отрицательные точки, проверка нечётности tg(−x) = −tg(x);

±π/2 — граничные значения, в которых tg(x) не определён — функция должна выбрасывать ArithmeticException;

3π/2 — значение, приводимое к π/2 после нормализации, должно выбрасывать ArithmeticException;

0.001, 0.01, 0.05, 0.1 — малые значения, при которых tg(x) ≈ x (линейное приближение, первый член ряда);

0.1, 0.3, 0.5, 0.7, 1.0, −0.1, −0.5, −1.0 — значения внутри области сходимости для сравнения с Math.tan();

0.3 + 10π, −0.3 − 10π — большие значения для проверки нормализации угла (периодичность);

Double.NaN, Double.POSITIVE_INFINITY — специальные значения, должны вызывать исключения;

Таблица тестовых данных для «интересных» точек

x	tg(x) ожидаемое	Точность	Примечание
0.0	0.0	1e-10	Точный ноль
π/6 ≈ 0.5236	1/√3 ≈ 0.5774	1e-6	Табличное значение
π/4 ≈ 0.7854	1.0	1e-3	Табличное значение
π/3 ≈ 1.0472	√3 ≈ 1.7321	1e-3	Близко к границе сходимости
−π/6	−1/√3	1e-6	Нечётность
−π/4	−1.0	1e-3	Нечётность
π/2	ArithmeticException	—	Не определён
−π/2	ArithmeticException	—	Не определён
3π/2	ArithmeticException	—	Нормализуется к π/2
0.1	Math.tan(0.1)	1e-6	Сравнение с эталоном
0.3	Math.tan(0.3)	1e-5	Сравнение с эталоном
0.5	Math.tan(0.5)	1e-4	Сравнение с эталоном
0.7	Math.tan(0.7)	1e-3	Сравнение с эталоном
1.0	Math.tan(1.0)	1e-2	Сравнение с эталоном
−0.1	Math.tan(−0.1)	1e-6	Отрицательные

x	tg(x) ожидаемое	Точность	Примечание
−0.5	Math.tan(−0.5)	1e-4	Отрицательные
−1.0	Math.tan(−1.0)	1e-2	Отрицательные
0.3 + 10π	Math.tan(0.3)	1e-3	Периодичность
−0.3 − 10π	Math.tan(−0.3)	1e-3	Периодичность

2. Помимо «особенных» точек, тестируется:

- **Сходимость ряда** — при увеличении числа членов ряда от 1 до 11 ошибка должна монотонно убывать. Проверяется для x = 0.5.
- **Проверка отдельных членов ряда:**
  - 1 член: $tg(x) \approx x$
  - 2 члена: $tg(x) \approx x + x^3/3$
  - 3 члена: $tg(x) \approx x + x^3/3 + 2x^5/15$
- **Свойства функции:**
  - Нечётность: $tg(-x) = -tg(x)$ для $x \in \{0.1, 0.3, 0.5, 0.7, 1.0, 1.2\}$
  - Периодичность: $tg(x + \pi) = tg(x)$, $tg(x + 2\pi) = tg(x)$
- **Обработка ошибок:**
  - terms = 0 → IllegalArgumentException
  - terms = −1 → IllegalArgumentException
  - terms = 12 (> maxTerms) → IllegalArgumentException

Результаты

Функция tg(x) корректно обрабатывает все тестовые случаи, включая граничные и специальные значения. Все результаты совпадают с эталонной реализацией Math.tan с заданной точностью. Функция правильно выбрасывает ArithmeticException для значений, в которых tg(x) не определён, и корректно нормализует углы с учётом периодичности. Сходимость ряда подтверждена — увеличение количества членов монотонно уменьшает ошибку.

Задание 2: *HashTable*

Эталонная реализация: http://www.cs.usfca.edu/~galles/visualization/OpenHash.html

Хеш-таблица с закрытой адресацией (метод цепочек / open hashing) — это структура данных, в которой каждый слот (бакет) таблицы содержит связанный список элементов. При возникновении коллизии (два ключа имеют одинаковый хеш) новый элемент добавляется в цепочку соответствующей корзины.
Хеш-функция: hash(key) = key mod capacity (с использованием Math.floorMod для корректной обработки отрицательных ключей).
Поддерживаемые операции:

- **insert(key, value)** — вставка пары ключ-значение (или обновление значения, если ключ уже существует)
- **search(key)** — поиск значения по ключу
- **delete(key)** — удаление элемента по ключу

Характерные точки алгоритма

Для тестирования выделены следующие характерные точки внутри алгоритма:

Точка	Обозначение	Описание
A	HASH_COMPUTED	Хеш вычислен, определён индекс корзины
B	BUCKET_EMPTY	Корзина пуста (null)
C	BUCKET_NOT_EMPTY	Корзина не пуста, содержит цепочку
D	KEY_FOUND	Ключ найден в цепочке
E	KEY_NOT_FOUND	Ключ не найден после обхода цепочки
F	ENTRY_INSERTED	Новый элемент вставлен в таблицу
G	ENTRY_UPDATED	Значение существующего элемента обновлено
H	ENTRY_DELETED	Элемент удалён из таблицы

Точка	Обозначение	Описание
I	SEARCH_RESULT_RETURNED	Результат поиска возвращён вызывающему коду

Эталонные последовательности прохождения характерных точек

Для каждой операции составлены эталонные последовательности попадания в характерные точки в зависимости от состояния таблицы:

Операция INSERT

Сценарий	Исходные данные	Эталонная последовательность
Вставка в пустую корзину	capacity=7, insert(10, 100)	A → B → F
Вставка с коллизией (новый ключ)	insert(1, 10), затем insert(8, 80) [8%7=1]	A → C → E → F
Обновление существующего ключа	insert(5, 50), затем insert(5, 500)	A → C → D → G

Операция SEARCH

Сценарий	Исходные данные	Эталонная последовательность
Поиск в пустой корзине	пустая таблица, search(10)	A → B → I
Ключ найден	insert(5, 50), search(5)	A → C → D → I
Ключ не найден в непустой корзине	insert(1, 10), search(22) [22%7=1]	A → C → E → I

Операция DELETE

Сценарий	Исходные данные	Эталонная последовательность
Удаление из пустой корзины	пустая таблица, delete(10)	A → B → E

Сценарий	Исходные данные	Эталонная последовательность
Ключ найден и удалён	insert(5, 50), delete(5)	A → C → D → H
Ключ не найден в непустой корзине	insert(1, 10), delete(22) [22%7=1]	A → C → E

Подход к тестированию

Будем проверять реализацию хеш-таблицы следующим образом:

- Для каждого публичного метода структуры (insert/search/delete — интерфейс совпадает с эталонной реализацией) выделяем основные сценарии работы метода (то есть возможные ветвления);
- Проверяем приватные методы как «белый ящик» — через вызов публичных методов с заданными исходными данными, которые обеспечивают выполнение проверяемого участка кода;
- Для каждого сценария записываем последовательность попадания в характерные точки и сравниваем с эталонной через механизм TraceListener.

Составленные тесты

Класс: InsertTest (операция INSERT)

- **testInsertIntoEmptyTable**: Вставка в пустую таблицу — проверяется размер и корректность поиска вставленного элемента.
- **testInsertWithoutCollisions**: Вставка нескольких элементов без коллизий — проверяется независимость корзин.
- **testInsertWithCollision**: Вставка элементов с коллизией (ключи 1 и 8 при capacity=7, оба дают хеш 1) — проверяется, что оба элемента в одной корзине.
- **testInsertUpdateExisting**: Обновление значения при повторной вставке — размер не изменяется, значение обновляется.
- **testInsertNegativeKey**: Вставка отрицательного ключа — проверяется корректность floorMod.
- **testInsertZeroKey**: Вставка ключа 0 — граничное значение.

Класс: SearchTest (операция SEARCH)

- **testSearchInEmptyTable**: Поиск в пустой таблице возвращает null.

- **testSearchExistingKey**: Поиск существующего элемента возвращает его значение.
- **testSearchNonExistingKey**: Поиск несуществующего элемента возвращает null.
- **testSearchInChain**: Поиск каждого элемента в цепочке из 3 элементов (ключи 1, 8, 15 — все в корзине 1).
- **testSearchMissingKeyInNonEmptyBucket**: Поиск отсутствующего ключа 22 (22%7=1) в непустой корзине с ключом 1.

Класс: DeleteTest (операция DELETE)

- **testDeleteFromEmptyTable**: Удаление из пустой таблицы возвращает false.
- **testDeleteExistingKey**: Удаление существующего элемента — размер уменьшается, поиск возвращает null.
- **testDeleteNonExistingKey**: Удаление несуществующего элемента — размер не изменяется.
- **testDeleteFromChain**: Удаление среднего элемента из цепочки — остальные элементы сохраняются.
- **testDeleteLastInBucket**: Удаление последнего элемента в корзине — корзина становится пустой.
- **testDeleteMissingKeyInNonEmptyBucket**: Удаление отсутствующего ключа из непустой корзины.

Класс: InsertTraceTest (трассировка INSERT)

- **testInsertEmptyBucketTrace**: Проверка последовательности A → B → F.
- **testInsertCollisionNewKeyTrace**: Проверка последовательности A → C → E → F.
- **testInsertUpdateTrace**: Проверка последовательности A → C → D → G.

Класс: SearchTraceTest (трассировка SEARCH)

- **testSearchEmptyBucketTrace**: Проверка последовательности A → B → I.
- **testSearchFoundTrace**: Проверка последовательности A → C → D → I.
- **testSearchNotFoundTrace**: Проверка последовательности A → C → E → I.

Класс: DeleteTraceTest (трассировка DELETE)

- **testDeleteEmptyBucketTrace**: Проверка последовательности A → B → E.
- **testDeleteFoundTrace**: Проверка последовательности A → C → D → H.
- **testDeleteNotFoundInChainTrace**: Проверка последовательности A → C → E.

Класс: HashFunctionTest (хеш-функция)

- **testHashPositive**: hash(10) = 10 % 7 = 3.
- **testHashZero**: hash(0) = 0.
- **testHashNegative**: hash(−3) = Math.floorMod(−3, 7) = 4 (а не −3).
- **testCollisionKeys**: hash(1) = hash(8) = hash(15) = 1.

Класс: BoundaryTest (граничные случаи)

- **testCapacityOne**: Ёмкость 1 — все элементы в одной корзине.
- **testInvalidCapacityZero**: Ёмкость 0 → IllegalArgumentException.
- **testInvalidCapacityNegative**: Ёмкость −5 → IllegalArgumentException.
- **testInvalidBucketIndex**: Индексы −1 и 7 → IndexOutOfBoundsException.
- **testClear**: Очистка таблицы — размер 0, все элементы удалены.
- **testContainsKey**: Проверка метода containsKey.
- **testMultipleInsertDelete**: 100 вставок, 50 удалений — проверка целостности.

Класс: ComplexTraceTest (комплексные сценарии)

- **testFullLifecycleTrace**: Полный жизненный цикл элемента: вставка → поиск → обновление → удаление → поиск удалённого. Для каждого шага проверяется эталонная последовательность характерных точек.
- **testChainSearchTrace**: Цепочка из 3 элементов — поиск существующего и несуществующего ключа с проверкой трассировки.

Результаты

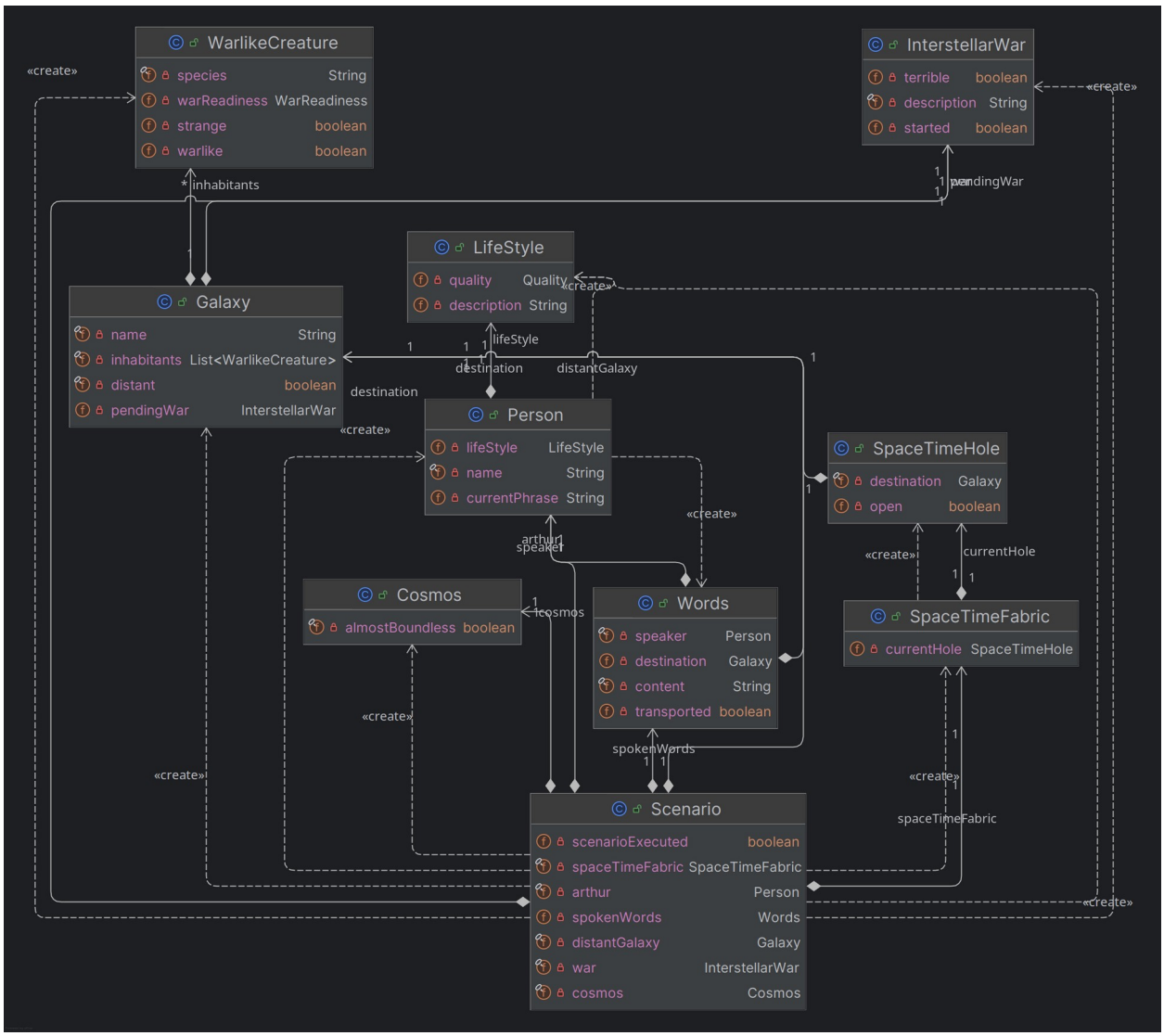
Как мы можем видеть, все тесты проходят успешно. Реализация хеш-таблицы корректна:

- Все операции (insert, search, delete) работают правильно.
- Коллизии разрешаются корректно через цепочки.

- Последовательности прохождения характерных точек совпадают с эталонными.
- Граничные случаи (ёмкость 1, отрицательные ключи, пустая таблица) обрабатываются корректно.
- Покрытие кода: строки — **98.0%**, ветви — **100.0%**.

Задание 3: *Описание предметной области*

Для начала, спроектируем нашу доменную область в соответствии с заданным текстом.



Спроектированная UML диаграмма предметной области.

Доменная модель

Разработанные классы:

Класс	Описание	Из текста
Person	Персонаж (Артур)	«Артур произнёс»
LifeStyle	Образ жизни (нормальный / проблемный / критический)	«большие проблемы с образом жизни»
Words	Произнесённые слова, которые могут путешествовать	«перенесла его слова»
SpaceTime Fabric	Ткань пространства-времени	«в ткани пространства-времени»
SpaceTime Hole	Случайная дыра в пространстве-времени	«открылась случайная дыра»
Cosmos	Космос (почти бескрайний)	«почти бескрайние просторы космоса»
Galaxy	Далёкая галактика	«в далёкую галактику»
WarlikeCr eature	Странное воинственное существо	«странные воинственные существа»
Interstellar War	Ужасная межзвёздная война	«на грани ужасной межзвёздной войны»
Scenario	Объединяет все элементы и выполняет описанный сценарий	Весь текст целиком

Составленные тесты

Класс: PersonTest (Персонаж)

- **testCreation**: Проверяет корректность создания объекта Person (имя и начальное состояние — фраза null).
- **testSayPhrase**: Проверяет, что метод sayPhrase корректно создаёт объект Words с содержанием, ссылкой на автора и обновляет текущую фразу персонажа.

- **testDefaultLifeStyle**: Проверяет, что у персонажа по умолчанию нормальный образ жизни (NORMAL).
- **testSetProblematicLifeStyle**: Проверяет, что можно установить проблемный образ жизни и метод hasProblems возвращает true.

Класс: LifeStyleTest (Образ жизни)

- **testNormalLifeStyle**: Нормальный образ жизни — нет проблем, описание «обычный образ жизни».
- **testProblematicLifeStyle**: Проблемный — есть проблемы, описание «большие проблемы с образом жизни».
- **testCriticalLifeStyle**: Критический — есть проблемы.
- **testChangeQuality**: Изменение качества образа жизни обновляет описание.

Класс: WordsTest (Произнесённые слова)

- **testCreation**: Проверяет создание слов с содержанием и автором; по умолчанию не перенесены (isTransported=false, destination=null).
- **testTransport**: Проверяет перенос слов в галактику — isTransported=true, destination равна указанной галактике.

Класс: SpaceTimeFabricTest (Ткань пространства-времени)

- **testInitialState**: Изначально нет дыр (hasOpenHole=false, getCurrentHole=null).
- **testOpenHole**: Открытие случайной дыры — дыра не null, открыта, ведёт в указанную галактику.
- **testCloseHole**: Закрытие дыры — hasOpenHole=false.
- **testCloseWhenNoHole**: Закрытие при отсутствии дыры не вызывает исключение.

Класс: SpaceTimeHoleTest (Дыра в пространстве-времени)

- **testCreation**: Создание открытой дыры с указанной галактикой назначения.
- **testTransportWords**: Перенос слов через дыру — слова помечаются как перенесённые, destination = галактика.
- **testTransportThroughClosedHole**: Попытка переноса через закрытую дыру → IllegalStateException.
- **testClose**: Закрытие дыры — isOpen=false.

Класс: GalaxyTest (Далёкая галактика)

- **testCreation**: Создание далёкой галактики — имя, isDistant=true, нет существ.
- **testAddInhabitants**: Добавление воинственных существ — hasWarlikeCreatures=true, размер списка = 1.
- **testOnBrinkOfWar**: Галактика на грани войны (война не начата) — isOnBrinkOfWar=true.
- **testNotOnBrinkOfWarWhenStarted**: Война уже началась — isOnBrinkOfWar=false.
- **testNoWar**: Галактика без войны — isOnBrinkOfWar=false.
- **testGetInhabitantsReturnsCopy**: getInhabitants возвращает копию списка — модификация копии не влияет на оригинал.

Класс: WarlikeCreatureTest (Воинственные существа)

- **testCreation**: Создание странного воинственного существа — isStrange=true, isWarlike=true, isOnBrinkOfWar=true.
- **testOnBrinkOfWar**: Готовность к войне — WarReadiness.ON_BRINK_OF_WAR.
- **testChangeWarReadiness**: Изменение состояния готовности (AT_WAR → не на грани, PEACEFUL → не на грани).
- **testSetStrange**: Существо может стать не странным.
- **testSetNotWarlike**: Существо может стать мирным.

Класс: InterstellarWarTest (Межзвёздная война)

- **testCreation**: Создание ужасной межзвёздной войны — описание, isTerrible=true, isStarted=false.
- **testStart**: Война начинается — isStarted=true.
- **testNotTerrible**: Война может быть не ужасной — isTerrible=false.

Класс: CosmosTest (Космос)

- **testAlmostBoundless**: Космос почти бескрайний — isAlmostBoundless=true.

Класс: ScenarioTest (Полный сценарий)

- **testInitialState**: Проверяет начальное состояние всех элементов сценария до выполнения — Артур с проблемным образом жизни, далёкая галактика со странными существами на грани войны, война ужасная и не начата.

- **testExecuteScenario**: Проверяет выполнение полного сценария — Артур произносит фразу, слова переносятся в далёкую галактику через дыру, дыра закрывается.
- **testGalaxyInhabitants**: Проверяет, что галактика населена странными воинственными существами, балансирующими на грани войны (2 существа, каждое isStrange=true, isWarlike=true, isOnBrinkOfWar=true).
- **testCosmosInScenario**: Космос доступен в сценарии и почти бескрайний.
- **testWarState**: Война ужасная, ещё не началась, галактика на грани.

Результаты

Как мы можем видеть, все тесты проходят успешно:

- Все объекты доменной модели создаются корректно.
- Состояния изменяются в соответствии с действиями.
- Сценарий выполняется последовательно — Артур произносит фразу, дыра открывается, слова переносятся, дыра закрывается.
- Инварианты сохраняются (существа странные и воинственные, война ужасная, космос почти бескрайний).
- Покрытие кода: строки — **93.5%**, ветви — **95.2%**.

Summary:

Total tests : 111

Failures : 0

Errors : 0

Skipped : 0

Time : 0.69s

TPO Lab 1

TPO Lab 1

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
ru.itmo.tpo.task3		88 %		95 %	10	84	10	153	9	73	0	12
ru.itmo.tpo.task1		91 %		83 %	3	12	3	21	1	6	0	1
ru.itmo.tpo.task2		97 %		100 %	2	35	2	100	2	18	0	3
Total	83 of 1 087	92 %	3 of 67	95 %	15	131	15	274	12	97	0	16

Итоговое тестовое покрытие.

Выводы

Во время выполнения лабораторной работы мы углубили свои знания в JUnit5, научились писать юнит-тесты, использовать параметризированные тесты и тесты на проверку составленной объектной модели.